EVOLVE

ACADEMY

Guide technique

Architecture applicative et paiement Chargily Pay v2



Lisez ce guide avant de modifier le code ou de soumettre une Pull Request.

EV-2026-002

Groupe REV · Division Éducation et Technologie · Octobre 2026 · Version 1.0.0

AVANT DE COMMENCER

Section 0 sur 14 : cadrage et principes directeurs

1 **Projet public** Next.js 15 (evolve-web)	2 **Supabase RLS** Auth et base partagée	3 **Dashboard Flutter** Staff CMS	4 **Chargily Pay** Passerelle CIB / Edahabia

1. **Application Web (evolve-web)** : Next.js 15 SSR hébergé sur Vercel. Portail étudiant public.
2. **Postgres et Auth (Supabase)** : Stockage partagé, sécurisé par Row Level Security (RLS).
3. **Dashboard Staff (Flutter)** : CMS d'administration propriétaire qui pilote la création des données.
4. **Passerelle Chargily Pay v2** : Orchestrateur national de paiement monétique algérien.

À SAVOIR ABSOLUMENT

Le schéma SQL appartient au dashboard Flutter : ne jamais écrire de migrations SQL dans evolve_school.

Les secrets API ne figurent JAMAIS côté client : seuls NEXT_PUBLIC_* sont exposés au navigateur.

Le support RTL (Arabe) est obligatoire : interdiction formelle des classes pl-* et pr-*.

CONTRIBUTING.md Lignes 165-185

```
167: ### The database is not ours
169: The schema lives in the `evolve_academy_dashboard` repository and is enforced
170: by row-level security in Postgres. This app reads it as the signed-in user - or
171: as `anon` for a visitor - and there is no service-role client anywhere. That is
172: what makes it safe to query straight from a Server Component without an
173: authorization check in the component: the check already happened, in the
174: database.
176: **Do not write migrations here.** If you need a column, a table, or a policy
177: change, raise it with a maintainer and it goes through the dashboard repository.
178: Working around a missing column in the frontend is the one shortcut that will
179: get a pull request closed rather than reviewed.
181: ## Arabic
183: Arabic is right-to-left, and it is a first-class language here rather than a
184: translation added at the end.
```

SOMMAIRE

Table des matières générale du référentiel

1. **Partie I : Config (p. 4 à 14)** : Architecture, configuration et internationalisation. Structure Next.js 15, guards CI, Biome, RTL.
2. **Partie II : Data et pages (p. 15 à 25)** : Base Supabase et routes applicatives. 4 clients Supabase, catalogue, salle de classe, communauté.
3. **Partie III : UI et lib (p. 26 à 33)** : Composants UI et logique métier. Matrice des composants, globe 3D, moteur de progression.
4. **Partie IV : Paiement (p. 34 à 46)** : Chargily Pay v2, cœur du système. Flux, checkout API, webhook HMAC, sécurité, tests.
5. **Partie V : Annexes (p. 47 à 52)** : Sécurité, déploiement Vercel et index exhaustif. Audits, FAQ des erreurs, index des fichiers.

ORGANISATION DE LA LECTURE

Une page = un sujet : chaque fiche synthétise schéma, responsabilités et extrait de code réel.

Le paiement Chargily Pay est couvert sur 13 pages dédiées (25 % du document).

L'annexe finale référence les fichiers du projet (hors répertoires ignorés de compilation).

docs/guide-technique/index.html Structure

1. Vue d'ensemble et Architecture (P.4-6)
2. Arborescence détaillée (P.7-8)
3. Configuration et Guards (P.9-12)
4. Internationalisation et RTL (P.13-14)
5. Supabase et politiques RLS (P.15-17)
6. Pages et routes app/ (P.18-25)
7. Composants interface (P.26-28)
8. Logique métier et lib/ (P.29-31)
9. Fonctionnalités et parcours (P.32-33)
10. PAIEMENT CHARGILY PAY (P.34-46)
11. Sécurité applicative (P.47)
12. Déploiement Vercel (P.48)
13. Débogage et commandes (P.49)
14. Audit et index exhaustif (P.50-52)

ARCHITECTURE GLOBALE

Section 1 sur 14 : flux de données et responsabilités

1 **Navigateur** React 19 Server/Client	2 **Next.js 15 (SSR)** App Router / Vercel Edge	3 **Supabase RLS** PostgreSQL authentifié	4 **Chargily Pay** API v2 et webhook

1. **Client Web** : Composants React 19 hydratés, gestion du thème et interactions locales.
2. **Serveur Next.js 15** : App Router, Server Components privilégiés pour les requêtes sans fuite.
3. **Base Supabase** : Accès direct filtré par jeton JWT de l'étudiant via Row-Level Security.
4. **Passerelle Chargily** : Initialisation des sessions d'achat et notification serveur par webhook.

PRINCIPES ARCHITECTURAUX

Zéro appel direct du client vers Chargily : toute transaction passe par la route serveur app/api/checkout.

Le serveur Next.js n'a pas besoin de vérifier les droits de lecture : Supabase RLS s'en charge en base.

Déploiement sur Vercel avec découpage optimal entre Server Components et Edge.

CONTRIBUTING.md Lignes 140-160

```
140: app/          routes, pages, layouts    may import components/ and lib/
141: components/   presentation              may import lib/, never app/
142: lib/          data and plumbing         imports neither
143:   data/       queries - one file per table, all `server-only`
144:   supabase/   the three clients: server, browser, middleware
145:   env.ts      the only file that reads process.env
146: i18n/         locale routing and message loading
147: messages/     fr.json - ar.json - en.json
148: scripts/      the guards
152: `scripts/boundary_guard.sh` enforces the arrows. The direction is the point: a
153: data function that imports a component is the start of a cycle.
156: ### The one page to read first
158: `app/[locale]/disciplines/page.tsx` is the reference implementation.
```

DÉPENDANCES DE PRODUCTION

Section 1 sur 14 : analyse de package.json

1 **Next.js 15** ^15.5.4 · Framework	2 **React 19** ^19.2.8 · Rendu SSR	3 **Supabase SSR** @supabase/ssr ^0.8.0
4 **next-intl** ^4.3.12 · FR / AR / EN	5 **Three.js** ^0.186.0 · Globe 3D	

1. **next et react** : Framework web et moteur de rendu : Server Actions et Server Components.
2. **@supabase/ssr** : Gestion d'authentification SSR : sessions utilisateur via cookies chiffrés.
3. **next-intl** : Internationalisation : routage localisé, négociation de locale, messages JSON.
4. **three** : Rendu WebGL : animation interactive du globe terrestre de la page Hero.

BONNES PRATIQUES

server-only interdit l'import de fonctions serveur critiques dans les composants client.

tailwind-merge et clsx garantissent des classes Tailwind dynamiques sans conflits CSS.

lucide-react fournit les icônes vectorielles légères, adaptées au rendu multi-directionnel.

package.json Lignes 16-28

```
16: "dependencies": {
17:   "@supabase/ssr": "^0.8.0",
18:   "@supabase/supabase-js": "^2.58.0",
19:   "clsx": "^2.1.1",
20:   "lucide-react": "^0.545.0",
21:   "next": "^15.5.4",
22:   "next-intl": "^4.3.12",
23:   "react": "^19.2.8",
24:   "react-dom": "^19.2.8",
25:   "server-only": "^0.0.1",
26:   "tailwind-merge": "^3.3.1",
27:   "three": "^0.186.0"
28: },
```

OUTILLAGE ET SCRIPTS DE BUILD

Section 1 sur 14 : configuration du développement

1	2	3	4
Biome @biomejs/biome ^2.2.0	**Tailwind v4** tailwindcss ^4.1.13	**TypeScript** ^5.9.2 (strict)	**Supabase CLI** supabase ^2.117.0

1. **Biome** : Formatage et analyse statique. Remplace ESLint et Prettier.
2. **Tailwind CSS v4** : Configuration native PostCSS avec variables logiques CSS.
3. **TypeScript 5.9** : Mode strict : types any interdits, vérification exhaustive.
4. **pnpm v10** : Gestionnaire de paquets, verrouillage via pnpm-lock.yaml.

CONSEILS D'EXÉCUTION

pnpm run lint vérifie la conformité du code sans le modifier.

pnpm run format réapplique le formatage officiel du projet.

Le script guards lance localement la suite complète des vérificateurs CI.

package.json Lignes 6-15

```
 6: "scripts": {
 7:   "dev": "next dev",
 8:   "build": "next build",
 9:   "start": "next start",
10:   "typecheck": "tsc --noEmit",
11:   "lint": "biome check .",
12:   "lint:fix": "biome check --write .",
13:   "format": "biome format --write .",
14:   "guards": "bash scripts/local_ci.sh"
15: },
```

ARBORESCENCE RACINE ET APP

Section 2 sur 14 : cartographie du répertoire app

1 **app/[locale]/** Routes publiques localisées	2 **app/api/** Routes serveur (checkout, webhooks)	3 **Racine** next.config.ts, biome.json, Makefile

1. **app/[locale]/** : Toutes les vues visibles, organisées sous le segment de langue.
2. **app/api/** : Endpoints REST : paiement, état de santé, synchronisation.
3. **app/layout.tsx et page.tsx** : Document racine, polices et redirections de base.
4. **middleware.ts** : Délègue la langue à next-intl et gère les sessions Supabase.

RÈGLE DU DOSSIER APP

Les routes API ne sont pas préfixées par [locale] : URLs prévisibles pour Chargily.

middleware.ts actualise silencieusement les cookies de session Supabase.

Toute nouvelle route utilisateur va dans app/[locale]/.

middleware.ts Lignes 1-16

```
1: import type { NextRequest } from "next/server";
2: import createIntlMiddleware from "next-intl/middleware";
3:
4: import { routing } from "@/i18n/routing";
5: import { updateSession } from "@/lib/supabase/middleware";
6:
7: const handleI18n = createIntlMiddleware(routing);
8:
9: export async function middleware(request: NextRequest) {
10:   const response = handleI18n(request);
11:
12:   return await updateSession(request, response);
13: }
14:
15: export const config = {
16:   matcher: ["/((?!api|_next|_vercel|.*\\..*).*)"],
```

COMPONENTS, LIB ET MESSAGES

Section 2 sur 14 : organisation modulaire du code

1 **components/** Présentation (max 300 lignes)	2 **lib/** Data et plomberie	3 **messages/ et scripts/** Dictionnaires JSON, guards CI

1. **components/** : Composants réutilisables. N'importent jamais depuis app/. Max 300 lignes.
2. **lib/data** : Accès aux données : un fichier par table. Tous marqués 'server-only'.
3. **messages/** : fr.json, ar.json, en.json : toutes les chaînes localisées.
4. **scripts/** : Scripts de garde : vérification des règles architecturales.

SÉGRÉGATION DES RÔLES

Un fichier de lib/ n'importe jamais un composant visuel.

Côté client, les modifications passent par les Server Actions ou l'API.

messages/ conserve une stricte équivalence de clés entre langues.

lib/data/categories.ts Lignes 1-25

```
1: import "server-only";
2: import { createClient } from "@/lib/supabase/server";
3: export interface Discipline {
4:   id: string;
5:   title: string;
6:   description: string | null;
7:   image_url: string | null;
8: }
9:
10: export async function getDisciplines(): Promise<Discipline[]> {
11:   const supabase = await createClient();
12:   const { data, error } = await supabase
13:     .from("categories")
14:     .select("id, name, description, image_url")
15:     .order("name", { ascending: true });
16:   if (error) {
17:     throw new Error(`Could not read disciplines: ${error.message}`);
18:   }
19:   return (data ?? []).map(
```

CONFIGURATION DU RUNTIME

Section 3 sur 14 : compilateur et plugins

1 **next.config.ts** Plugin next-intl et headers	2 **tsconfig.json** Alias @/* et strict	3 **postcss.config.mjs** Intégration Tailwind v4

1. **next.config.ts** : Encapsule createNextIntlPlugin pour injecter les requêtes de traduction.
2. **tsconfig.json** : Alias @/* pointant vers la racine du projet.
3. **biome.json** : Règles du linter et formateur.
4. **postcss.config.mjs** : Plugin @tailwindcss/postcss pour le traitement CSS.

POINTS DE CONTRÔLE

L'alias @/ évite les chemins relatifs fragiles (../../).

Next.js applique des en-têtes de sécurité stricts (HSTS, Content-Type Options, X-Frame-Options).

TypeScript est en noEmit : le packaging est délégué à Turbopack.

next.config.ts Lignes 1-30

```
1: import type { NextConfig } from "next";
2: import createNextIntlPlugin from "next-intl/plugin";
3:
4: const withNextIntl = createNextIntlPlugin("./i18n/request.ts");
5:
6: const securityHeaders = [
7:   { key: "X-DNS-Prefetch-Control", value: "on" },
11:   { key: "Strict-Transport-Security",
13:     value: "max-age=63072000; includeSubDomains; preload" },
16:   { key: "X-Frame-Options", value: "DENY" },
20:   { key: "X-Content-Type-Options", value: "nosniff" },
24:   { key: "Referrer-Policy",
25:     value: "strict-origin-when-cross-origin" },
28:   { key: "Permissions-Policy",
29:     value: "camera=(), microphone=(), geolocation=(), browsing-topics=()" },
```

VARIABLES D'ENVIRONNEMENT

Section 3 sur 14 : protection et accès via lib/env.ts

1. .env.local	2. lib/env.ts	3. Application
Ignoré par git · dev local	Seul fichier lisant process.env	Importe { env } depuis @/lib/env

1. **.env.local** : Secrets locaux. Ne doit JAMAIS être versionné.
2. **lib/env.ts** : Point d'accès unique : valide et expose en typage strict, erreur si absente.
3. **secret_guard.sh et boundary_guard.sh** : Empêchent tout commit de clé et process.env hors de env.ts.
4. **NEXT_PUBLIC_*** : Exposées au navigateur : uniquement l'URL Supabase et la clé Anon publique.

RÈGLE CRITIQUE DE SÉCURITÉ

CHARGILY_SECRET_KEY ne doit JAMAIS porter le préfixe NEXT_PUBLIC_ : fuite fatale sinon.

Il est interdit d'écrire process.env.* dans un composant.

Si une variable obligatoire manque au démarrage, env.ts lève une exception explicite.

lib/env.ts Lignes 40-68

```
42: get chargilyMode(): "test" | "live" {
43:   const mode = process.env.CHARGILY_MODE?.toLowerCase();
44:   return mode === "live" ? "live" : "test";
45: },
46: get chargilySecretKey(): string {
47:   return process.env.CHARGILY_SECRET_KEY
48:     || process.env.CHARGILY_API_KEY || "";
50: },
56: get chargilyApiUrl(): string {
57:   if (process.env.CHARGILY_API_URL) {
58:     return process.env.CHARGILY_API_URL;
59:   }
60:   return this.chargilyMode === "live"
61:     ? "https://pay.chargily.net/api/v2"
62:     : "https://pay.chargily.net/test/api/v2";
63: },
64: get siteUrl(): string {
66:   return process.env.NEXT_PUBLIC_SITE_URL || process.env.APP_URL
68:     || "https://evolve-academy.dz"
```

LES GUARDS D'ASSURANCE QUALITÉ

Section 3 sur 14 : automatisation du contrôle CI

1 **Secret guard** Détection de fuites	2 **Boundary guard** Frontières d'import	3 **Design token guard** Interdit pl-* / pr-*	4 **Component size** Plafond 300 lignes

1. **secret_guard.sh** : Bloque tout commit contenant des motifs de clés privées ou mots de passe.
2. **boundary_guard.sh** : Vérifie les sens d'importation entre app, components et lib.
3. **design_token_guard.sh** : Refuse les couleurs hexadécimales brutes et les marges physiques gauche/droite.
4. **component_size_guard.sh** : Échoue si un fichier .tsx dépasse 300 lignes.

CONFORMITÉ OBLIGATOIRE

Les guards ne sont pas des suggestions : la CI échoue si l'un d'eux est rouge.

Un composant proche de 250 lignes émet un avertissement.

Pour tout tester en local : make ci-local.

scripts/component_size_guard.sh Lignes 15-34

```
15: set -uo pipefail
16: cd "$(dirname "${BASH_SOURCE[0]}")/.."
19: LIMIT=300
20: WARN=250
21: fail=0
23: while IFS= read -r file; do
24:   lines=$(wc -l < "$file" | tr -d ' ')
25:   if [ "$lines" -gt "$LIMIT" ]; then
26:     echo "[SIZE] $file - $lines lines (limit $LIMIT)"
27:     fail=1
28:   elif [ "$lines" -gt "$WARN" ]; then
29:     echo "[SIZE] $file - $lines lines, approaching the limit"
30:   fi
31: done < <(find app components -name '*.tsx' -type f)
33: exit $fail
```

PROTOCOLE DE CONTRIBUTION

Section 3 sur 14 : cycle Git et définition de terminé

1 **Branche locale** feat/* ou fix/*	2 **development** Branche d'intégration	3 **main (prod)** Release servie par Vercel

1. **Cycle de branches** : Départ depuis development. Jamais depuis main.
2. **Pull Requests Draft** : Ouvrir la PR dès le premier jour pour faciliter la relecture.
3. **Taille maximale** : Moins de 400 lignes modifiées : découper en lots digestes.
4. **Préfixes de revue** : blocking / suggestion / nit : évite de bloquer inutilement.

DÉFINITION DU TRAVAIL TERMINÉ

Pas de merge sans make ci-local au vert.

Toute modification d'interface est vérifiée sur mobile et en Arabe.

Ne jamais contourner un guard : adapter la règle publiquement.

CONTRIBUTING.md Lignes 25-48

```
26: fix/short-description --+
27: feat/another-thing -----+--> development --> main
28: chore/third-thing ------+    (integration)   (release)
31: | Branch        | Purpose                      | Merge method |
33: | main          | Release. What prod serves.   | Merge commit |
34: | development   | Integration. Lands here 1st. | Squash       |
35: | feat/* fix/*  | One change each.             | -            |
37: Branch from `development`, never from `main`.
41: You cannot push to `development` or `main`, force-push them, delete
42: them, or merge a red pull request into them. That is enforced by
43: organisation rulesets, not by convention.
46: This is meant to be freeing, not restrictive. You cannot break this
47: repository. Push early, push unfinished work, open draft pull requests
48: on day one of a task.
```

ARCHITECTURE NEXT-INTL

Section 4 sur 14 : configuration multilingue du site

1	2	3	4
Requête HTTP /fr /ar /en	**routing.ts** locales: fr, ar, en	**request.ts** getRequestConfig()	**messages/*.json** Dictionnaire chargé

1. **i18n/routing.ts** : Déclare fr, ar, en. Français par défaut (defaultLocale).
2. **i18n/request.ts** : Charge de façon asynchrone messages/${locale}.json.
3. **app/[locale]/layout.tsx** : Définit les attributs html lang et dir (ltr ou rtl).
4. **messages/*.json** : Chaînes regroupées par composants ou domaines.

GESTION DES LOCALES

Toute clé manquante dans une langue secondaire produit un avertissement au runtime.

La détection de la langue de l'utilisateur est gérée en amont par le middleware.

Le français est la langue de référence pour les clés initiales.

i18n/routing.ts Lignes 1-15

```
1: import { defineRouting } from "next-intl/routing";
3: /**
4:  * The three languages the academy operates in.
6:  * French is the default because it is the language of instruction and
7:  * of most incoming traffic. Arabic is not an afterthought: it is
8:  * right-to-left, and the whole layout has to survive `dir="rtl"`.
9:  * Check every UI change in Arabic before opening the pull request.
10:  */
11: export const routing = defineRouting({
12:   locales: ["fr", "ar", "en"],
13:   defaultLocale: "fr",
14: });
```

SUPPORT RTL ET NAVIGATION

Section 4 sur 14 : règles de rendu en langue arabe

1 Propriétés physiques pl-*, pr-*, left, right : INTERDIT	**2 Propriétés logiques** ps-*, pe-*, ms-*, me-* : VALIDE	**3 Link localisé** @/i18n/navigation : OBLIGATOIRE

1. **ps-* et pe-*** : Espacements logiques : s'inversent nativement en lecture arabe (RTL).
2. **ms-* et me-*** : Marges logiques : alignement correct des icônes et boutons.
3. **i18n/navigation.ts** : Réexporte Link, redirect et useRouter en conservant le préfixe de langue.
4. **next/link (interdit)** : Omet la locale et bascule l'utilisateur en français.

RÈGLE D'OR ARABE (RTL)

design_token_guard.sh stoppe tout commit contenant des classes pl- ou pr-.

Toujours tester l'affichage sous /ar/... avant de soumettre.

Les icônes directionnelles (flèches de retour) sont inversées en RTL.

i18n/navigation.ts Lignes 1-15

```
1: import { createNavigation } from "next-intl/navigation";
2: import { routing } from "./routing";
4: /**
5:  * Locale-aware replacements for Next's navigation primitives.
7:  * Import `Link` from here, never from `next/link`. Next's own `Link`
8:  * drops the locale segment, so a visitor reading the Arabic site is
9:  * silently moved to the French one by clicking a link.
10:  * `scripts/boundary_guard.sh` fails the build on a `next/link` import.
11:  */
12: export const { Link, redirect, usePathname, useRouter, getPathname } =
13:   createNavigation(routing);
```

MODÈLE DE DONNÉES SUPABASE

Section 5 sur 14 : schéma relationnel PostgreSQL

1 **courses** id, title, price, is_published	2 **lessons** id, course_id, video_url, is_free	3 **enrollments** user_id, course_id, payment_status
4 **workshops** id, title, price, video_url	5 **workshop_enrollments** user_id, status	

1. **courses et lessons** : Formations séquentielles de leçons vidéo avec indicateur is_free.
2. **enrollments** : Associe un étudiant à un cours ; stocke chargily_checkout_id et payment_status.
3. **workshops** : Sessions intensives payantes : replay vidéo et livrables de projet.
4. **workshop_enrollments** : Trace les paiements d'ateliers : pending, paid ou failed.

ARCHITECTURE DES DONNÉES

L'unicité est garantie par (user_id, course_id) et (user_id, workshop_id).

Le prix réel est lu depuis courses.price ou workshops.price lors du checkout.

Les leçons is_free = true sont visibles sans inscription payante.

lib/data/lesson-progress.ts Lignes 1-30

```
5: export async function saveLessonProgress(
6:   lessonId: string, progressPercentage: number,
9:   completed: boolean, lastPosition: number,
10: ) {
11:   const supabase = await createClient();
13:   const { data: { user } } = await supabase.auth.getUser();
17:   if (!user) {
18:     throw new Error("User not authenticated");
19:   }
21:   const { error } = await supabase.from("lesson_progress").upsert({
23:     user_id: user.id,
24:     lesson_id: lessonId,
25:     progress_percentage: completed ? 100 : progressPercentage,
26:     completed,
27:     last_position: Math.floor(lastPosition),
28:     updated_at: new Date().toISOString(),
29:   },
```

LES 4 CLIENTS SUPABASE

Section 5 sur 14 : isolation et stratégies de connexion

1	2	3	4
server.ts Server Components, cookies de session	**client.ts** Browser, createBrowserClient()	**middleware.ts** createServerClient, cookies	**admin.ts** Service role, server-only

1. **lib/supabase/server.ts** : Lit la session via les cookies Next.js. Utilisé dans toutes les pages.
2. **lib/supabase/client.ts** : Interactions légères côté client et abonnements temps réel.
3. **lib/supabase/middleware.ts** : Met à jour les tokens de session expirés.
4. **lib/supabase/admin.ts** : Outrepasse RLS pour les paiements et webhooks.

RÈGLE CRITIQUE D'ISOLATION

createAdminClient() est protégé par 'server-only' (commit 483076f).

Ne jamais importer admin.ts dans un composant 'use client'.

En production, le client admin utilise SUPABASE_SERVICE_ROLE_KEY.

lib/supabase/admin.ts Lignes 1-19

```
1: import "server-only";
2: import { createClient } from "@supabase/supabase-js";
3: import { env } from "@/lib/env";
5: export function createAdminClient() {
6:   const serviceRoleKey =
7:     env.supabaseServiceRoleKey || env.supabaseAnonKey;
8:   if (!serviceRoleKey) {
9:     throw new Error("Missing SUPABASE_SERVICE_ROLE_KEY ...");
11:   }
13:   return createClient(env.supabaseUrl, serviceRoleKey, {
14:     auth: {
15:       autoRefreshToken: false,
16:       persistSession: false,
17:     },
18:   });
19: }
```

SÉCURITÉ RLS ET POLITIQUES POSTGRES

Section 5 sur 14 : contrôle d'accès au niveau des lignes

1	2	3	4
Anonyme Cours publics et preview	**Étudiant** auth.uid() = user_id	**Formateur** teacher_id = auth.uid()	**Service role** Bypasse RLS : webhooks

1. **RLS sur courses** : Sélection publique si is_published = true.
2. **RLS sur lessons** : is_free = true lisible par tous ; sinon enrollment avec payment_status = 'paid'.
3. **RLS sur enrollments** : Un utilisateur ne lit que ses inscriptions (user_id = auth.uid()).
4. **RLS sur messages** : Lecture et écriture réservées aux participants (sender_id ou recipient_id).

PRINCIPE DE MOINDRE PRIVILÈGE

Le frontend n'embarque aucune vérification manuelle : Postgres filtre à la source.

Une manipulation d'ID en URL retourne null grâce aux politiques RLS.

Les politiques RLS sont administrées dans le dépôt evolve_academy_dashboard.

lib/data/workshop-enrollment.ts Lignes 1-32

```
6: export interface WorkshopEnrollment {
7:   id: string;
8:   payment_status: string;
9:   chargily_checkout_id: string | null;
10:  enrolled_at?: string | null;
11: }
13: export async function getWorkshopEnrollment(
14:   userId: string, workshopId: string,
16: ): Promise<WorkshopEnrollment | null> {
17:   const adminSupabase = createAdminClient();
18:   const { data, error } = await adminSupabase
19:     .from("workshop_enrollments")
20:     .select("id, payment_status, chargily_checkout_id, enrolled_at")
21:     .eq("user_id", userId)
22:     .eq("workshop_id", workshopId)
23:     .maybeSingle();
25:   if (error) throw new Error(`Unable to load ...: ${error.message}`);
29:   return data;
```

PAGE D'ACCUEIL ET STRUCTURE

Section 6 sur 14 : architecture du point d'entrée

1 **app/[locale]/page .tsx** Accueil SSR	2 **Hero.tsx** Canvas Three.js	3 **Continue learning** Reprise dernière leçon	4 **Navbar et Footer** Navigation multilingue

1. **app/[locale]/page.tsx** : Orchestre les sections publiques et les cours personnalisés.
2. **components/Hero.tsx** : Globe terrestre animé et preuves sociales d'Evolve Academy.
3. **ContinueLearningSection** : Progression courante si l'étudiant est identifié.
4. **EnrollmentPathSection** : Guide le visiteur à travers les formations et packs.

CONCEPTION DE LA HOMEPAGE

Les données sont chargées en parallèle côté serveur pour minimiser le FCP.

Le globe 3D est chargé dynamiquement côté client (pas d'erreur d'hydratation).

L'interface s'adapte à l'état de connexion de l'utilisateur.

app/[locale]/page.tsx Lignes 1-35

```
1: import { getTranslations, setRequestLocale } from "next-intl/server";
2: import Hero from "@/components/Hero";
3: import HorizontalCourseSection from "@/components/HorizontalCourseSection";
4: import ContinueLearningSection from "@/components/home/ContinueLearningSection";
5: import EnrollmentPathSection from "@/components/home/EnrollmentPathSection";
6: import IntroGate from "@/components/IntroGate";
7: import IntroPreloader from "@/components/IntroPreloader";
8: import Navbar from "@/components/Navbar";
20: export default async function HomePage({ params }) {
25:   const { locale } = await params;
26:   setRequestLocale(locale);
28:   const tHome = await getTranslations("home");
29:   const supabase = await createClient();
31:   let publicCourses = {
32:     beginnerCourses: [] as Course[],
33:     partnerCourses: [] as Course[],
34:     exclusiveCourses: [] as Course[],
35:     trendingCourses: [] as Course[],
```

CATALOGUE ET DISCIPLINES

Section 6 sur 14 : exploration pédagogique et filtrage

1 **formations/** FormationsBrowser.tsx	2 **disciplines/page.tsx** Référence d'implémentation	3 **components/formations/** Card, Curriculum, Sidebar

1. **FormationsBrowser.tsx** : Recherche textuelle, tri par niveau et filtrage thématique.
2. **disciplines/page.tsx** : Exemple cité dans CONTRIBUTING.md (SSR + RLS + RTL complet).
3. **FormationCard.tsx** : Titre, durée, badge de domaine et prix en Dinars (DZD).
4. **FormationsSidebar.tsx** : Sélection rapide des disciplines sur grand écran.

MODÈLE D'ARCHITECTURE

disciplines/page.tsx est le modèle à dupliquer pour toute nouvelle page de données.

Le nombre de leçons et d'heures est pré-agrégé dans la requête SQL initiale.

La recherche supporte la saisie en arabe avec le sens d'écriture.

app/[locale]/disciplines/page.tsx Lignes 1-35

```
4: /** --- THE TRACER ---
6:  * The one page in this repository that is finished, and the pattern
7:  * every other data-backed page copies. It goes all the way through:
10:  * route -> lib/data query -> Supabase client as the caller -> RLS in
11:  * Postgres -> rendered markup -> three languages -> right-to-left
13:  * Nothing here is mocked. The rows come from the same database the
14:  * Flutter dashboard writes to.
22:  * Note what is absent: no `useState`, no `useEffect`, no loading
23:  * spinner, no fetch in the browser. This is a Server Component.
28: /** Rendered per request, not at build time. It was
29:  * `export const revalidate = 3600` first. With static generation,
33:  * `next build` runs this query. CI has no database credentials, so
34:  * the query failed - and the build still reported success. */
```

DÉTAIL DU COURS ET ÉDITION

Section 6 sur 14 : parcours de consultation et prérequis

1 **courses/[id]/page.tsx** Vérification paiement et accès	2 **courses/[id]/checkout/** Prix et CheckoutButton	3 **courses/[id]/edit/** Interface formateur

1. **courses/[id]/page.tsx** : Programme détaillé, instructeur et état d'inscription de l'élève.
2. **checkout/page.tsx** : Redirige vers Chargily si la formation n'est pas acquise.
3. **edit/page.tsx** : Les enseignants réorganisent leçons et métadonnées.
4. **CourseHeroHeader et LessonsList** : Avancement global et liste interactive des chapitres.

VERROUILLAGE SÉQUENTIEL

Un non-inscrit est redirigé vers le tunnel checkout.

Le prérequis de la série doit être terminé à 100 %.

L'édition n'est permise qu'à l'auteur du cours (RLS).

app/[locale]/courses/[id]/page.tsx Lignes 76-112

```
77: const { data: seriesCourse } = await supabase
78:   .from("series_courses").select("series_id, order_index")
80:   .eq("course_id", id).maybeSingle();
84: if (seriesCourse && seriesCourse.order_index > 1) {
85:   const { data: previousCourse } = await supabase
86:     .from("series_courses").select("course_id")
88:     .eq("series_id", seriesCourse.series_id)
89:     .eq("order_index", seriesCourse.order_index - 1).maybeSingle();
92:   if (previousCourse) {
93:     const { data: previousLessons } = await supabase
94:       .from("lessons").select("id")
96:       .eq("course_id", previousCourse.course_id);
...
100:      const { data: completedLessons } = await supabase
101:        .from("lesson_progress").select("lesson_id")
103:        .eq("user_id", user.id).eq("completed", true)
105:        .in("lesson_id", previousLessonIds);
107:      if (completedLessons?.length !== previousLessons.length) {
108:        redirect(`/${locale}/formations`);
```

SALLE DE CLASSE ET LECTEUR VIDÉO

Section 6 sur 14 : expérience d'apprentissage interactive

1 **lessons/[lessonId]/page.tsx** Contrôle RLS de lecture	2 **LessonVideo.tsx** YouTube et Storage	3 **MarkLessonComplete.tsx** Met à jour lesson_progress

1. **lessons/[lessonId]/page.tsx** : Vérifie l'accès (leçon gratuite ou cours payé) et injecte les ressources.
2. **LessonVideo.tsx** : Lecteur avec sauvegarde périodique de la dernière position.
3. **MarkLessonComplete.tsx** : Marque la leçon terminée et déclenche le calcul global.
4. **AssignmentSection.tsx** : Téléchargement des énoncés et remise des exercices.

CONTRÔLE D'ACCÈS VIDÉO

Une leçon payante renvoie 403 si le paiement n'est pas validé.

last_position permet de reprendre exactement au même moment.

Toutes leçons complétées : le certificat est généré.

app/api/lesson-progress/route.ts Lignes 1-35

```
4: export async function POST(request: Request) {
5:   try {
6:     const supabase = await createClient();
8:     const { data: { user } } = await supabase.auth.getUser();
12:    if (!user) {
13:      return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
14:    }
16:    const body = await request.json();
18:    const { lessonId, progressPercentage, completed, lastPosition } = body;
20:    if (!lessonId) {
21:      return NextResponse.json({ error: "lessonId is required" },
23:        { status: 400 });
25:    }
27:    const percentage = Math.min(100, Math.max(0, Number(progressPercentage) || 0));
32:    const position = Math.max(0, Math.floor(Number(lastPosition) || 0));
34:    const isCompleted = Boolean(completed) || percentage >= 95;
```

ATELIERS ET SESSIONS PRATIQUES

Section 6 sur 14 : masterclasses et formations intensives

1 **ateliers/** AteliersBrowser.tsx	2 **ateliers/[id]/page.tsx** Fiche et inscription	3 **WorkshopJoinButton.tsx** Paiement atelier	4 **components/workshop/** Player, Submission, Certificate

1. **AteliersBrowser.tsx** : Catalogue des ateliers avec date de session et prix.
2. **ateliers/[id]/page.tsx** : Description, objectifs, formateurs et lecteur de session.
3. **WorkshopJoinButton.tsx** : Initie le paiement Chargily (workshopId).
4. **WorkshopVideoPlayer.tsx** : Replay accessible si workshop_enrollments.payment_status = 'paid'.

FONCTIONNEMENT DES ATELIERS

Le flux de paiement des ateliers est unifié avec celui des cours dans l'API checkout.

Les inscrits accèdent aux ressources exclusives et au salon d'échange.

Un atelier terminé permet de soumettre un projet pour validation du mentor.

app/[locale]/ateliers/[id]/page.tsx Lignes 1-35

```
1: import { ArrowLeft, Award, CheckCircle2, Clock3, Layers3 } from "lucide-react";
3: import { notFound } from "next/navigation";
4: import { getTranslations, setRequestLocale } from "next-intl/server";
5: import Footer from "@/components/Footer";
6: import Navbar from "@/components/Navbar";
11: import WorkshopEnrolledView from "@/components/workshop/WorkshopEnrolledView";
13: import { Link } from "@/i18n/navigation";
14: import {
15:   getWorkshopEnrollment,
16:   verifyChargilyWorkshopPayment,
17: } from "@/lib/data/workshop-enrollment";
19: import WorkshopJoinButton from "./WorkshopJoinButton";
21: type Props = {
22:   params: Promise<{ locale: string; id: string }>;
26:   searchParams: Promise<{ payment?: string }>;
29: };
```

ESPACE COMMUNAUTÉ ET SHOWCASE

Section 6 sur 14 : partage de projets et mentorat

1 **community/** Showcase des réalisations	2 **community/[id]/page.tsx** Likes, commentaires, démo	3 **SubmitProjectForm.tsx** Upload image et technologies

1. **community/page.tsx** : Galerie publique des réalisations étudiantes, filtre par outils.
2. **SubmitProjectForm.tsx** : Validation côté client et compression d'images avant upload.
3. **LikeButton et Comments** : Évaluation par les pairs et mentorat participatif.
4. **EditProjectForm.tsx** : L'auteur met à jour ses liens GitHub ou démo.

VALEUR COMMUNAUTAIRE

Les images sont compressées par lib/imageCompressor.ts avant envoi vers Supabase Storage.

Seul l'auteur ou un administrateur peut éditer ou supprimer (RLS).

La pagination assure des temps de réponse constants.

app/[locale]/community/SubmitProjectForm.tsx Lignes 1-35

```
1: "use client";
3: import { AlertCircle, CheckCircle2, FileText, Sparkles, Tag } from "lucide-react";
10: import { useTranslations } from "next-intl";
11: import ProjectImageUploader from "@/components/community/ProjectImageUploader";
12: import ProjectTechSuggestions from "@/components/community/ProjectTechSuggestions";
13: import ProjectUrlInputs from "@/components/community/ProjectUrlInputs";
14: import { useSubmitProjectForm } from "./useSubmitProjectForm";
16: export default function SubmitProjectForm() {
17:   const t = useTranslations("community");
18:   const {
19:     title, setTitle, description, setDescription,
23:     category, setCategory, technologies, setTechnologies,
27:     githubUrl, setGithubUrl, demoUrl, setDemoUrl,
31:     previewUrl, loading, message, addTech,
35:     handleImageChange,
```

MESSAGERIE INSTANTANÉE

Section 6 sur 14 : communication entre étudiants et encadrants

1 **messages/** MessagingClient.tsx	2 **api/messages/contacts/** Contacts légitimes uniquement	3 **MessageComposer et Thread** Composition, bulles temps réel

1. **MessagingClient.tsx** : Liste des conversations ouvertes et fil de discussion actif.
2. **/api/messages/contacts** : Profils partageant un cours ou un atelier avec l'étudiant.
3. **ConversationSidebar.tsx** : Messages non lus et horodatages des derniers échanges.
4. **MessageComposer.tsx** : Saisie multilingue, envoi immédiat (Entrée).

SÉCURITÉ DE LA MESSAGERIE

Impossible d'initier une conversation avec des inconnus hors contexte d'apprentissage.

Le taux de requêtes est surveillé pour éviter le spam.

Les messages sont protégés en base : aucun tiers ne lit les discussions privées.

app/api/messages/contacts/route.ts Lignes 1-35

```
1: import { NextResponse } from "next/server";
2: import { isStudentRole, isStudentTeacherPair, isTeacherRole }
6:   from "@/lib/community-directory";
7: import { createAdminClient } from "@/lib/supabase/admin";
8: import { createClient } from "@/lib/supabase/server";
10: export const dynamic = "force-dynamic";
12: export async function GET() {
13:   try {
14:     const supabase = await createClient();
15:     const { data: { user }, error: authError } =
18:       await supabase.auth.getUser();
20:     if (!user) {
21:       return NextResponse.json({ error: "Authentication required." },
24:         { status: 401 });
25:     }
26:     if (authError) throw authError;
28:     const { data: currentProfile, error: profileError } = await supabase
29:       .from("profiles").select("role").eq("id", user.id).maybeSingle();
```

TABLEAU DE BORD ET PROFIL

Section 6 sur 14 : suivi de carrière et expérience élève

1 **dashboard/** DashboardClient.tsx	2 **components/dashboard/** StatsGrid, CertificateModal, ResumeBanner	3 **profile/** ProfileForm.tsx

1. **DashboardClient.tsx** : Formations en cours, heures passées et certificats.
2. **DashboardStatsGrid.tsx** : Formations commencées, achevées et taux moyen de réussite.
3. **DashboardResumeBanner.tsx** : Formation la plus urgente à continuer en un clic.
4. **profile/page.tsx** : Préférences pour le moteur de recommandation.

SUIVI PÉDAGOGIQUE

Un visiteur non connecté voit DashboardGuestView.

Les attestations sont synchronisées avec lib/course-completion.ts.

Les modifications de profil réactualisent les recommandations.

app/[locale]/dashboard/page.tsx Lignes 1-35

```
1: import type { Metadata } from "next";
2: import { getTranslations } from "next-intl/server";
3: import DashboardGuestView from "@/components/dashboard/DashboardGuestView";
4: import type { DbEnrollment, DbLesson, DbProgress,
8:   EnrolledCourseItem, UserStats, WorkshopItem,
11: } from "@/components/dashboard/types";
14: import { createClient } from "@/lib/supabase/server";
15: import DashboardClient from "./DashboardClient";
23: export async function generateMetadata({ params }): Promise<Metadata> {
24:   const { locale } = await params;
25:   const t = await getTranslations({ locale,
26:     namespace: "dashboardUi.meta" });
28:   return {
29:     title: t("title"),
30:     description: t("description"),
31:     robots: { index: false, follow: false },
34:   };
```

MATRICE DES COMPOSANTS UI

Section 7 sur 14 : cartographie et répartition des composants

1 **Navbar et Footer** Layout global	2 **Hero et Globe 3D** Page d'accueil	3 **CheckoutButton** Courses et ateliers	4 **Course et Workshop** Cartes et lecteurs

1. **Navbar et Footer** : Utilisés dans app/[locale]/layout.tsx sur 100 % des pages.
2. **CheckoutButton.tsx** : Lance Chargily depuis courses/[id]/checkout et ateliers/[id].
3. **LockedCourseCard.tsx** : Teaser sécurisé : cadenas et incitation au déblocage.
4. **LessonVideo.tsx** : Lecteur utilisé uniquement dans la salle de classe active.

RÈGLES DES COMPOSANTS

Chaque composant UI respecte le plafond de 300 lignes.

Les composants de présentation n'importent jamais de fonctions serveur sensibles.

Le style Tailwind utilise uniquement des classes logiques adaptées au RTL.

components/CheckoutButton.tsx Lignes 1-35

```
1: "use client";
3: import { CreditCard, Loader2 } from "lucide-react";
4: import { useTranslations } from "next-intl";
5: import { useState } from "react";
7: export default function CheckoutButton({ courseId, locale, price }:
11:   { courseId: string; locale: string; price?: number }) {
16:   const t = useTranslations("siteUi.checkoutButton");
17:   const [loading, setLoading] = useState(false);
18:   const [error, setError] = useState("");
20:   async function handleCheckout() {
21:     try {
22:       setLoading(true);
23:       setError("");
25:       const response = await fetch("/api/checkout", {
26:         method: "POST",
27:         headers: { "Content-Type": "application/json" },
30:         body: JSON.stringify({ courseId, locale }),
34:       });
```

NAVIGATION ET STRUCTURE GLOBALE

Section 7 sur 14 : cadrage ergonomique et responsivité

1	2	3
Navbar.tsx DesktopNav et MobileNav	**Footer.tsx** Liens, mentions, réseaux	**Sélecteur de langue** FR / AR / EN

1. **Navbar.tsx** : Barre principale : Connexion ou Mon Espace selon l'authentification.
2. **DesktopNav et MobileNav** : Menus responsifs pour ordinateur et smartphone.
3. **Footer.tsx** : Catégories, disciplines et conditions générales de vente.
4. **BackgroundGlow.tsx** : Reflets de lumière et dégradés sombres caractéristiques d'Evolve.

ERGONOMIE GLOBALE

La Navbar détecte le défilement pour appliquer backdrop-blur.

Le changement de langue conserve la page courante, seul le préfixe change.

Le menu mobile s'ouvre de droite à gauche en arabe.

components/Navbar.tsx Lignes 1-35

```
1: "use client";
3: import { useLocale, useTranslations } from "next-intl";
4: import { useEffect, useState } from "react";
5: import { usePathname, useRouter } from "@/i18n/navigation";
6: import { createClient } from "@/lib/supabase/client";
7: import DesktopNav from "./navbar/DesktopNav";
8: import MobileNav from "./navbar/MobileNav";
11: let cachedProfile: Profile | null = null;
12: let cachedProfileLoaded = false;
14: export default function Navbar() {
20:   const [profile, setProfile] = useState(cachedProfile);
22:   const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
23:   const [langDropdownOpen, setLangDropdownOpen] = useState(false);
24:   const [scrolled, setScrolled] = useState(false);
27:   useEffect(() => {
28:     const handleScroll = () => setScrolled(window.scrollY > 20);
32:     window.addEventListener("scroll", handleScroll);
33:     return () => window.removeEventListener("scroll", handleScroll);
```

ANIMATION HERO 3D ET GLOBE

Section 7 sur 14 : expérience visuelle WebGL Three.js

1 **EarthGlobe.tsx** Canvas Three.js	2 **lib/earth.ts et earth-mask.ts** Shaders, maillage, masque	3 **IntroPreloader.tsx** Préchargement des textures

1. **EarthGlobe.tsx** : Globe 3D interactif, Afrique du Nord et Algérie mises en valeur.
2. **lib/earth.ts** : Scène, caméra perspective et lumières directionnelles.
3. **lib/earth-mask.ts** : Masquage des continents pour un rendu épuré.
4. **IntroPreloader.tsx** : Animation d'introduction pendant le chargement des ressources.

OPTIMISATION GRAPHIQUE

Le globe est désactivé sur les appareils à faible GPU.

Le composant est encapsulé dans dynamic(() => ..., { ssr: false }).

La rotation s'arrête quand la section n'est plus visible.

components/EarthGlobe.tsx Lignes 1-35

```
6: export default function EarthGlobe() {
7:   const containerRef = useRef<HTMLDivElement>(null);
9:   useEffect(() => {
10:    if (!containerRef.current) return;
12:    let earth: Awaited<ReturnType<typeof createEarth>> | null = null;
13:    let cancelled = false;
15:    createEarth(containerRef.current).then((instance) => {
16:      // React may have already cleaned up the effect.
17:      if (cancelled) {
18:        instance.dispose();
19:        return;
20:      }
22:      earth = instance;
23:      earth.play();
24:    });
26:    return () => {
27:      cancelled = true;
29:      earth?.dispose();
30:      earth = null;
31:    };
32:  }, []);
```

INFRASTRUCTURE ET LOGIQUE MÉTIER

Section 8 sur 14 : plomberie système et résilience réseau

1 **lib/apiClient.ts** Fetch et erreurs	2 **rateLimiter.ts** Anti-bruteforce	3 **lib/validations.ts** Schémas, sanitization	4 **lib/logger.ts** Masquage des PII

1. **lib/apiClient.ts** : En-têtes et codes d'erreur HTTP standardisés.
2. **lib/rateLimiter.ts** : Protège checkout et messages contre les abus.
3. **lib/validations.ts** : Valide e-mails, montants et payloads avant toute requête base.
4. **lib/logger.ts** : Journalise sans jamais divulguer jetons ou mots de passe.

RÈGLES DE SÉCURITÉ RÉSEAU

Le rate limiter accorde 10 tentatives par minute au tunnel checkout.

Toute entrée utilisateur est nettoyée contre XSS et injection SQL.

En production, les logs critiques vont vers la console Vercel.

lib/rateLimiter.ts Lignes 1-35

```
1: interface RateLimitRecord { count: number; resetAt: number; }
6: const rateLimitStore = new Map<string, RateLimitRecord>();
8: // Periodic cleanup of expired records every 5 minutes
9: if (typeof setInterval !== "undefined") {
10:   setInterval(() => {
11:     const now = Date.now();
12:     for (const [key, record] of rateLimitStore.entries()) {
13:       if (now > record.resetAt) rateLimitStore.delete(key);
16:     }
17:   }, 300_000);
18: }
27: export function checkRateLimit(
28:   identifier: string,
29:   limit = 60,
30:   windowMs = 60_000,
31: ): RateLimitResult {
32:   const now = Date.now();
33:   const record = rateLimitStore.get(identifier);
35:   if (!record || now > record.resetAt) {
```

MOTEUR DE PROGRESSION ET CERTIFICATS

Section 8 sur 14 : logique d'achèvement pédagogique

1 **lesson-progress.ts** completed, last_position	2 **course-completion.ts** 100 % = certificat	3 **CertificateModal** Attestation numérique

1. **lib/data/lesson-progress.ts** : Complétion et pourcentage de suivi de chaque leçon.
2. **lib/course-completion.ts** : Compare les leçons validées au total des leçons publiées.
3. **CourseCertificateSection.tsx** : Certificat officiel dès 100 % de progression.
4. **DashboardCertificateModal.tsx** : Consultation et export du diplôme.

PROCESSUS DE CERTIFICATION

Le recalcul intervient à chaque validation de chapitre.

Un cours de zéro leçon ne délivre pas de certification.

Le nom du certificat vient en priorité du profil vérifié.

lib/course-completion.ts Lignes 1-35

```
1: import { createClient } from "@/lib/supabase/client";
3: type CompleteLessonParams = {
4:   lessonId: string;  courseId: string;
6:   nextCourseId?: string | null;  isLastCourse?: boolean;
8:   formationId?: string | null;   locale: string;
10: };
12: export async function completeLessonAndNavigate({ lessonId, courseId,
13:   nextCourseId, isLastCourse, formationId, locale }: CompleteLessonParams) {
20:   const supabase = createClient();
22:   const { data: { user } } = await supabase.auth.getUser();
26:   if (!user) return;
28:   // 1. Mark current lesson as completed
29:   const { error: progressError } = await supabase
30:     .from("lesson_progress")
31:     .upsert({
33:       user_id: user.id,
34:       lesson_id: lessonId,
35:       progress_percentage: 100,
```

RECOMMANDATIONS ET PROFILS

Section 8 sur 14 : personnalisation du parcours apprenant

1 **learning-profile.ts** Intérêts, objectif, niveau	2 **recommendations.ts** Matching tags et niveau	3 **log-search.ts** Analytics sans tracking invasif

1. **lib/data/learning-profile.ts** : Objectifs professionnels et domaines de prédilection.
2. **lib/data/recommendations.ts** : Croise profil et cours disponibles pour suggérer le prochain module.
3. **log-search.ts** : Mesure les besoins les plus demandés pour orienter les créations.
4. **community-directory.ts** : Profils publics des étudiants et formateurs certifiés.

ALGORITHME D'AFFINITÉ

Il privilégie les formations non encore commencées.

Les données d'analyse ne contiennent aucune donnée personnelle nominative.

Le profil peut être mis à jour à tout moment depuis les réglages.

lib/data/learning-profile.ts Lignes 1-35

```
5: export async function getLearningProfile() {
6:   const supabase = await createClient();
8:   const { data: { user } } = await supabase.auth.getUser();
12:  if (!user) {
13:    return null;
14:  }
16:  // 1. Get enrolled courses
17:  const { data: enrollments, error: enrollmentError } = await supabase
18:    .from("enrollments")
19:    .select("course_id, enrolled_at")
20:    .eq("user_id", user.id)
21:    .order("enrolled_at", { ascending: false });
25:  if (enrollmentError) throw new Error(enrollmentError.message);
29:  const courseIds =
30:    enrollments?.map((enrollment) => enrollment.course_id) ?? [];
32:  if (courseIds.length === 0) {
33:    return { enrolledCourses: [], completedCourses: [],
```

CYCLE D'APPRENTISSAGE

Section 9 sur 14 : séquencement pédagogique et diplômation

1 **1. Inscription**	2 **2. Visionnage**	3 **3. Exercice**
4 **4. Validation**	5 **5. Diplôme**	

1. **Inscription** : Paiement ou accès gratuit : débloque les leçons protégées dans Supabase.
2. **Visionnage** : Lecteur avec mémorisation ; progression dans lesson_progress.
3. **Devoirs pratiques** : Remise d'assignations ; évaluation formative par le formateur.
4. **Séquencement** : L'accès à la leçon suivante dépend de l'achèvement de la précédente.

PARCOURS PÉDAGOGIQUE

Un étudiant ne saute pas une leçon requise sans valider la précédente.

Le taux global se met à jour en temps réel sur la barre du cours.

En cas d'interruption, le lecteur reprend à la seconde mémorisée.

components/lesson/useLessonProgress.ts Lignes 1-35

```
5: interface UseLessonProgressOptions {
6:   lessonId: string;  youtubeUrl: string;  courseId: string;
9:   nextCourseId?: string | null;  isLastCourse?: boolean;
11:  formationId?: string | null;   locale: string;
13: }
15: /** Custom hook to initialize YouTube player and track video playback
16:  *  progress in Supabase. Handles automatic completion marking and
17:  *  redirection to next lesson / formation completion. */
19: export function useLessonProgress({ lessonId, youtubeUrl, courseId,
24:   nextCourseId, isLastCourse, formationId, locale }) {
28:   const containerRef = useRef<HTMLDivElement>(null);
29:   const playerRef = useRef<YouTubePlayer | null>(null);
30:   const redirectedRef = useRef(false);
32:   useEffect(() => {
33:     const videoId = getYouTubeId(youtubeUrl);
34:     if (!videoId || !containerRef.current) return;
```

ATTESTATIONS ET PROJETS D'ATELIERS

Section 9 sur 14 : validation pratique et certification

1 **Dépôt du projet** Figma, GitHub ou démo	2 **Validation mentor** Évaluation par le formateur	3 **WorkshopCertificate** Certificat avec badge

1. **WorkshopProjectSubmission.tsx** : Envoi du travail pour revue personnalisée.
2. **WorkshopProjectHook** : Statuts : soumis, en cours de révision, validé.
3. **WorkshopCertificateSection.tsx** : Attestation de participation et de réussite.
4. **WorkshopIncludedFeatures.tsx** : Rappel : replay, fichiers sources, réseau alumni.

VALIDATION DES MASTERCLASSES

L'attestation mentionne le volume horaire réel et le formateur référent.

Les projets validés se partagent en un clic sur l'espace communautaire.

Le replay reste accessible sans limite de durée pour l'étudiant ayant payé.

components/workshop/WorkshopProjectSubmission.tsx Lignes 1-35

```
1: "use client";
3: import { CheckCircle2, ExternalLink, FolderGit2, Globe,
9:   Loader2, Send } from "lucide-react";
11: import { Link } from "@/i18n/navigation";
12: import type { WorkshopProjectItem } from "./types";
13: import { useWorkshopProjectSubmission } from "./useWorkshopProjectSubmission";
15: interface WorkshopProjectSubmissionProps {
16:   workshopId: string;
17:   workshopTitle: string;
18:   workshopDomain: string | null;
19:   existingProject?: WorkshopProjectItem | null;
20:   userId: string;
21: }
23: export default function WorkshopProjectSubmission({ workshopId,
24:   workshopTitle, workshopDomain, existingProject, userId }) {
30:   const { t, project, isEditing, setIsEditing, title,
```

PAIEMENT : VUE D'ENSEMBLE DU FLUX

Section 10.1 sur 14 : synopsis du paiement Edahabia et CIB

1. Étudiant	2. Next.js API	3. Chargily Pay	4. Webhook et base
Clic Débloquer	POST /api/checkout, prix lu en BDD	Page Edahabia / CIB, carte + OTP	payment_status = 'paid'

1. **Interface utilisateur** : L'étudiant clique sur Participer ou Débloquer. Authentification vérifiée.
2. **Serveur Next.js (/api/checkout)** : Lit le montant réel dans Supabase, crée l'inscription en 'pending'.
3. **Passerelle Chargily Pay** : Redirige vers SATIM (Edahabia / CIB) avec code SMS OTP.
4. **Notification webhook** : Chargily notifie le serveur de façon infalsifiable : accès débloqué.

RÈGLE FONDAMENTALE DE SÉCURITÉ

Le client ne communique JAMAIS avec l'API Chargily : tout transite par le serveur Next.js.

Le montant est obligatoirement extrait de PostgreSQL.

L'accès définitif n'est libéré que par le webhook validé cryptographiquement.

app/api/checkout/route.ts — Lignes 270-299

```
270: const controller = new AbortController();
271: const timeoutId = setTimeout(() => controller.abort(), 10_000);
273: const origin = new URL(request.url).origin;
275: const checkoutResponse = await fetch(`${apiUrl}/checkouts`, {
276:   method: "POST",
277:   headers: { Authorization: `Bearer ${secretKey}`,
279:     "Content-Type": "application/json" },
281:   signal: controller.signal,
282:   body: JSON.stringify({
283:     amount: course.price,
284:     currency: "dzd",
285:     locale: locale === "ar" || locale === "fr" ? locale : "en",
286:     description: course.title,
287:     success_url: `${origin}/${locale}/courses/${course.id}?payment=success`,
290:     failure_url: `${origin}/${locale}/courses/${course.id}/checkout?payment=failed`,
293:     webhook_endpoint: `${env.appUrl || origin}/api/webhooks/chargily`,
294:     metadata: { user_id: user.id, course_id: course.id },
```

RÉPARTITION DES RESPONSABILITÉS

Section 10.1 sur 14 : qui fait quoi dans la transaction

1 Navigateur	2 Serveur Next.js	3 Chargily	4 Base Supabase
Envoie courseId / workshopId	Auth, prix, plafonds, HMAC	Transaction SATIM, webhook signé	'pending' puis 'paid' via admin

1. **Navigateur client** : Zéro confiance monétaire : fournit seulement l'identifiant du produit et la langue.
2. **Serveur Next.js** : Valide supabase.auth.getUser() et contacte l'API Chargily v2 en POST.
3. **Passerelle Chargily** : Encaisse les fonds ; renvoie la confirmation avec signature HMAC en en-tête.
4. **Base Supabase** : Traçabilité complète : horodatage et clé de réconciliation.

SÉGRÉGATION DES POUVOIRS

Aucune coordonnée bancaire (numéro de carte, CVC) ne transite ni n'est stockée sur nos serveurs.

Le client ne peut modifier ni le montant en DZD ni l'URL du webhook.

La tentative est enregistrée dès la création de session, statut 'pending'.

app/api/checkout/route.ts Lignes 12-36

```
12: export async function POST(request: Request) {
13:   try {
14:     const supabase = await createClient();
16:     const { data: { user } } = await supabase.auth.getUser();
20:     if (!user) {
21:       return NextResponse.json(
22:         { error: "Authentification requise pour effectuer un paiement." },
23:         { status: 401 });
25:     }
27:     // Rate Limiting per user for checkout (10 requests per minute)
28:     const rateLimit = checkRateLimit(
29:       `checkout:${user.id}`,
30:       RATE_LIMIT_TIERS.CHECKOUT_PAYMENT.limit,
31:       RATE_LIMIT_TIERS.CHECKOUT_PAYMENT.windowMs,
32:     );
34:     if (!rateLimit.allowed) {
35:       return NextResponse.json(
```

CONFIGURATION : MODES TEST ET LIVE

Section 10.2 sur 14 : différences d'environnements et clés

1 **Mode test (sandbox)** pay.chargily.net/test/api/v2, CHARGILY_MODE = test	**2** **Mode live (production)** pay.chargily.net/api/v2, CHARGILY_MODE = live

1. **CHARGILY_SECRET_KEY** : Clé privée : authentifie les requêtes et calcule le HMAC. CÔTÉ SERVEUR SEULEMENT.
2. **CHARGILY_MODE** : 'test' en local et staging, 'live' en production.
3. **CHARGILY_API_URL** : Calculée par lib/env.ts selon CHARGILY_MODE.
4. **NEXT_PUBLIC_SITE_URL** : URL publique : URLs de retour et adresse du webhook.

SÉCURITÉ DES CLÉS PRIVÉES

CHARGILY_SECRET_KEY ne figure jamais dans un fichier tracké ni avec NEXT_PUBLIC_.

En mode test (/test/api/v2), les clés commencent généralement par test_sk_.

Sur Vercel, les variables de production sont injectées dans Environment Variables.

lib/env.ts Lignes 42-63

```
42: get chargilyMode(): "test" | "live" {
43:   const mode = process.env.CHARGILY_MODE?.toLowerCase();
44:   return mode === "live" ? "live" : "test";
45: },
46: get chargilySecretKey(): string {
47:   return process.env.CHARGILY_SECRET_KEY
48:     || process.env.CHARGILY_API_KEY || "";
50: },
51: get chargilyApiKey(): string { ... },
56: get chargilyApiUrl(): string {
57:   if (process.env.CHARGILY_API_URL) {
58:     return process.env.CHARGILY_API_URL;
59:   }
60:   return this.chargilyMode === "live"
61:     ? "https://pay.chargily.net/api/v2"
62:     : "https://pay.chargily.net/test/api/v2";
63: },
```

MODÈLE DE DONNÉES DU PAIEMENT

Section 10.3 sur 14 : tables, relations et états

1 Table enrollments (cours) user_id, course_id (unique), payment_status	**2 Table workshop_enrollments** user_id, workshop_id (unique), payment_amount, chargily_checkout_id, paid_at

1. **chargily_checkout_id** : Reçu lors du POST /checkouts. Clé de réconciliation du webhook.
2. **payment_status** : Cycle de vie : 'pending', 'paid' ou 'failed'.
3. **paid_at** : Horodatage renseigné à la validation du webhook ou de la vérification.
4. **payment_amount** : Montant réglé en DZD pour l'audit comptable et les plafonds.

INTÉGRITÉ RELATIONNELLE

Une contrainte d'unicité interdit deux lignes pour un même utilisateur et même cours.

createAdminClient est requis pour mettre à jour chargily_checkout_id.

Les politiques RLS interdisent à un étudiant d'écrire son propre payment_status.

app/api/checkout/route.ts Lignes 320-355

```
320: // Pre-create enrollment with pending status using admin client
321: const { error: enrollmentError } = isWorkshop
322:   ? await adminSupabase.from("workshop_enrollments").upsert({
324:       user_id: user.id, workshop_id: course.id,
326:       payment_status: "pending", payment_amount: course.price,
328:       payment_method: "chargily",
329:       chargily_checkout_id: checkout.id,
330:       enrolled_at: new Date().toISOString(),
332:     }, { onConflict: "user_id,workshop_id" })
334:   : await adminSupabase.from("enrollments").upsert({
337:       user_id: user.id, course_id: course.id,
338:       payment_status: "pending", ...
343:     }, { onConflict: "user_id,course_id" });
346: if (enrollmentError) {
347:   console.error("[Checkout] Unable to persist pending enrollment:", ...);
351:   return NextResponse.json({ error: ... }, { status: 500 });
```

INTERFACE ET CHECKOUTBUTTON

Section 10.4 sur 14 : expérience visuelle du déclencheur

1 **État 1 : initial** Bouton avec prix	2 **État 2 : chargement** Désactivé, spinner	3 **État 3 : redirection** window.location.href = checkout_url

1. **CheckoutButton.tsx** : Reçoit courseId ou workshopId ; gère loading et l'appel POST.
2. **Gestion des erreurs** : Alertes localisées : session expirée, déjà inscrit, serveur saturé.
3. **Désactivation préventive** : Anti-double-clic : bouton bloqué dès le premier clic.
4. **i18n des libellés** : 'Débloquer', 'Paiement en cours', 'Redirection' en FR, AR, EN.

UX DU PAIEMENT

Si l'utilisateur n'est pas connecté, redirection vers /sign-in.

En cas d'erreur 409 (déjà payé), le bouton passe en 'Accéder au contenu'.

L'appel transmet { courseId, locale } ou { workshopId, locale }.

components/CheckoutButton.tsx Lignes 40-75

```
41:     throw new Error(t("initError"));
42:   }
44:   const checkoutUrl = data.checkout_url;
46:   if (!checkoutUrl) {
47:     throw new Error(t("missingUrl"));
48:   }
50:   window.location.href = checkoutUrl;
51: } catch (err) {
52:   console.error(err);
53:   setError(err instanceof Error ? err.message : t("error"));
54:   setLoading(false);
55: }
...
60: <button type="button" onClick={handleCheckout} disabled={loading}
66:   {loading ? (
68:     <Loader2 className="h-5 w-5 animate-spin" />
69:     <span>{t("redirecting")}</span>
72:   ) : ( <CreditCard .../> <span>{price ...</span> )}
```

CRÉATION DU CHECKOUT (API ROUTE)

Section 10.5 sur 14 : logique serveur de POST /api/checkout

1. Authentification	2. Prix en base	3. Appel Chargily	4. Retour
getUser(), 401 si absent	Lecture courses / workshops	POST /checkouts, metadata	{ checkout_url, id } · 200

1. **Contrôle d'identité** : Erreur 401 Unauthorized si aucun utilisateur valide.
2. **Lecture du tarif en BDD** : Aucun prix n'est accepté du client.
3. **Requête vers Chargily v2** : Montant en DZD, description, URLs de redirection, endpoint webhook.
4. **Enregistrement pending** : Upsert avec chargily_checkout_id pour réconciliation.

PROTECTION FINANCIÈRE SERVEUR

Timeout strict de 10 secondes via AbortController.

metadata contient l'ID utilisateur et l'ID du cours.

Inscription déjà payée : 409 Conflict immédiat.

app/api/checkout/route.ts — Lignes 85-126

```
85: // 1. Prevent Duplicate Payment / Subscription
86: let existingPaymentStatus: string | null = null;
87: if (isWorkshop) {
88:   const existingEnrollment = await getWorkshopEnrollment(user.id, workshopId);
92:   existingPaymentStatus = existingEnrollment?.payment_status ?? null;
93: } else {
94:   const { data: existingEnrollment, error: enrollmentLookupError } =
95:     await supabase.from("enrollments")
97:       .select("id, payment_status")
98:       .eq("user_id", user.id).eq("course_id", courseId).maybeSingle();
102:  if (enrollmentLookupError) { ... status: 500 }
112:  existingPaymentStatus = existingEnrollment?.payment_status ?? null;
113: }
115: if (existingPaymentStatus === "paid") {
116:   return NextResponse.json({ error: "Vous etes deja inscrit ...",
121:     alreadyEnrolled: true }, { status: 409 });
125: }
```

PLAFONDS DE SÉCURITÉ ET ANTI-ABUS

Section 10.5 sur 14 : garde-fous et limites monétaires

1	2	3
Max par transaction MAX_TRANSACTION_AMOUNT_DZD : 50 000 DZD	**Max utilisateur / 24 h** MAX_DAILY_USER_SPENDING_DZD : 150 000 DZD	**Rate limit checkout** 10 requêtes / minute

1. **Plafond par transaction** : Bloque tout achat unitaire au-delà de 50 000 DZD.
2. **Plafond quotidien glissant** : Cumul cours et ateliers sur 24 h ; blocage au-delà de 150 000 DZD.
3. **Limiteur de fréquence** : 10 tentatives de checkout par minute (429 Too Many Requests).
4. **Audit des dépenses** : Interroge enrollments et workshop_enrollments sur 24 h.

DISPOSITIFS ANTI-FRAUDE

Ces plafonds empêchent l'exploitation d'une faille de prix ou l'abus d'une carte volée.

Le limiteur renvoie X-RateLimit-Limit et X-RateLimit-Reset.

Le cumul quotidien compte cours ET ateliers.

app/api/checkout/route.ts Lignes 165-205

```
165: // 2. Spending Cap Validation
166: if (course.price > MAX_TRANSACTION_AMOUNT_DZD) {
167:   return NextResponse.json({ error: `Le montant (${course.price} DZD)
169:     depasse le plafond ...` }, { status: 400 });
173: }
175: const adminSupabase = createAdminClient();
176: const yesterday = new Date(Date.now() - 86_400_000).toISOString();
177: const { data: recentPayments, error: recentPaymentsError } =
178:   await supabase.from("enrollments").select("payment_amount")
180:     .eq("user_id", user.id).eq("payment_status", "paid")
182:     .gte("enrolled_at", yesterday);
195: const { data: recentWorkshopEnrollments, error: workshopPaymentsError } =
196:   await adminSupabase.from("workshop_enrollments")
198:     .select("workshop_id")
199:     .eq("user_id", user.id).eq("payment_status", "paid")
201:     .not("chargily_checkout_id", "is", null)
202:     .gte("enrolled_at", yesterday);
```

LE WEBHOOK CHARGILY

Section 10.6 sur 14 : réception et analyse de la notification

1 **Chargily server** Événement et signature	2 **POST /api/webhooks/chargily** request.text() brut, HMAC-SHA256	3 **Supabase admin** payment_status, paid_at = now()

1. **request.text()** : Corps lu en texte brut avant tout parsing JSON, pour valider la signature.
2. **Header 'signature'** : Empreinte calculée par Chargily avec notre clé secrète.
3. **Authentification webhook** : 401 si la signature est absente ou invalide.
4. **Admin client** : Pas de session utilisateur : createAdminClient() requis.

RÈGLE VITALE DU WEBHOOK

Ne jamais se fier uniquement à la page de retour : le webhook est le seul arbitre.

Il doit répondre 200 OK pour accuser réception.

En cas d'échec, Chargily réessaie à intervalles espacés.

app/api/webhooks/chargily/route.ts Lignes 1-24

```
1: import crypto from "node:crypto";
2: import { NextResponse } from "next/server";
3: import { env } from "@/lib/env";
4: import { createAdminClient } from "@/lib/supabase/admin";
6: export async function POST(request: Request) {
7:   try {
8:     const secretKey = env.chargilySecretKey;
10:    if (!secretKey) {
11:      return NextResponse.json(
12:        { error: "Chargily configuration is missing" },
13:        { status: 500 });
15:    }
17:    const body = await request.text();
19:    const signature = request.headers.get("signature");
21:    if (!signature) {
22:      return NextResponse.json({ error: "Missing signature" },
23:        { status: 401 });
```

VALIDATION DE LA SIGNATURE HMAC

Section 10.6 sur 14 : sécurité cryptographique du webhook

1	2	3
Corps brut + clé createHmac('sha256')	**timingSafeEqual** Temps constant	**Validation ou rejet** Identique : suite · Sinon 401

1. **crypto.createHmac** : Empreinte SHA-256 du corps brut avec CHARGILY_SECRET_KEY.
2. **crypto.timingSafeEqual** : Comparaison à temps constant, contre les attaques par timing.
3. **Vérification de longueur** : Garde : signature.length === expectedSignature.length avant les buffers.
4. **Rejet immédiat** : 401 sans toucher à la base si l'empreinte diffère.

PROTECTION CRYPTOGRAPHIQUE

Ne jamais comparer une signature avec ===.

Le corps ne subit aucune altération d'espaces ou d'encodage avant le hachage.

Sans CHARGILY_SECRET_KEY, la route échoue proprement en 500.

app/api/webhooks/chargily/route.ts — Lignes 25-39

```
25: const expectedSignature = crypto
26:   .createHmac("sha256", secretKey)
27:   .update(body)
28:   .digest("hex");
30: if (
31:   signature.length !== expectedSignature.length ||
32:   !crypto.timingSafeEqual(
33:     Buffer.from(signature),
34:     Buffer.from(expectedSignature),
35:   )
36: ) {
37:   return NextResponse.json({ error: "Invalid signature" },
38:     { status: 401 });
39: }
```

IDEMPOTENCE ET GESTION DES STATUTS

Section 10.6 sur 14 : traitement des événements paid et failed

1	2	3
Événement 'paid' payment_status = 'paid'	**Failed / canceled / expired** payment_status = 'failed'	**Garantie d'idempotence** neq('payment_status','paid')

1. **Mise à jour simultanée** : Promise.all sur enrollments ET workshop_enrollments via chargily_checkout_id.
2. **Garde anti-écrasement** : neq('payment_status','paid') : un 'failed' tardif ne rétrograde pas un accès payé.
3. **Vérification de présence** : Au moins une ligne modifiée, sinon 404.
4. **Idempotence pure** : Recevoir 2 ou 5 fois le même webhook donne le même état.

ROBUSTESSE DU TRAITEMENT

Un paiement validé n'est JAMAIS écrasé par une annulation ultérieure.

paid_at est horodaté à l'heure exacte de la notification.

Promise.all réduit le traitement à moins de 80 ms.

app/api/webhooks/chargily/route.ts Lignes 44-91

```
44: if (["failed", "canceled", "expired"].includes(checkout.status)) {
45:   const failedClient = createAdminClient();
46:   await Promise.all([
47:     failedClient.from("enrollments")
49:       .update({ payment_status: "failed" })
50:       .eq("chargily_checkout_id", checkout.id)
51:       .neq("payment_status", "paid"),
52:     failedClient.from("workshop_enrollments") ... (idem)
57:   ]);
58:   return NextResponse.json({ received: true, status: checkout.status });
59: }
61: if (checkout.status !== "paid") { return ... { received: true } }
77: const supabase = createAdminClient();
79: const [courseResult, workshopResult] = await Promise.all([
80:   supabase.from("enrollments")
82:     .update({ payment_status: "paid", paid_at: new Date().toISOString() })
83:     .eq("chargily_checkout_id", checkoutId).select("id"),
85:   supabase.from("workshop_enrollments") ... (idem)
90: ]);
```

PAGES DE RETOUR ET SECOURS

Section 10.7 sur 14 : redirection client et vérification active

1 **Succès : ?payment=success** Redirection cours / atelier, bannière verte	2 **Échec : ?payment=failed** Retour checkout, message d'échec	3 **Secours** verifyChargilyPayment(), polling si webhook retardé

1. **success_url** : /courses/[id]?payment=success ou /ateliers/[id]?payment=success.
2. **failure_url** : Permet de réitérer le paiement sans perdre le contexte.
3. **verifyChargilyPayment** : Si l'utilisateur arrive avant le webhook, interroge directement l'API Chargily.
4. **Feedback utilisateur** : Bannières : transaction confirmée ou carte refusée.

DOUBLE CONTRÔLE D'ACCÈS

?payment=success dans l'URL ne débloque RIEN sans validation en BDD.

verifyChargilyPayment interroge GET /checkouts/${id} avec le secret serveur.

Si Chargily confirme 'paid', la BDD est mise à jour immédiatement.

lib/data/course-payment.ts Lignes 6-40

```
6: export async function verifyChargilyPayment(
7:   enrollmentId: string, chargilyCheckoutId: string,
9: ): Promise<boolean> {
10:   try {
11:     if (!env.chargilySecretKey || !chargilyCheckoutId) return false;
13:     const res = await fetch(
14:       `${env.chargilyApiUrl}/checkouts/${chargilyCheckoutId}`,
15:       { headers: { Authorization: `Bearer ${env.chargilySecretKey}` },
19:         cache: "no-store" });
23:     if (res.ok) {
24:       const data = await res.json();
25:       if (data.status === "paid") {
26:         const adminSupabase = createAdminClient();
27:         const { error } = await adminSupabase.from("enrollments")
29:           .update({ payment_status: "paid", paid_at: ... })
30:           .eq("id", enrollmentId);
38:         return true;
```

DÉVERROUILLAGE EFFECTIF DES ACCÈS

Section 10.8 sur 14 : disponibilité des leçons et certificats

1 **Leçons de formation** isPaid === true : vidéos et supports	**2** **Replay d'atelier** isEnrolled === true : lecteur débloqué	**3** **Attestation et diplôme** Certificat accessible

1. **Accès aux leçons** : courses/[id] charge les leçons avec leurs vidéos dès que isPaid est confirmé.
2. **Accès aux ateliers** : workshop_enrollments rend accessible le lecteur et le dépôt de projet.
3. **Politique en cas de litige** : 'failed' ou suppression de l'inscription coupe l'accès (RLS).
4. **Traçabilité des droits** : Les formations débloquées figurent sur le tableau de bord.

CONTRÔLE D'ACCÈS PERSISTANT

Aucune URL de vidéo privée n'est envoyée au client sans paiement en base.

Recharger la page conserve l'état débloqué (persistance PostgreSQL).

Les ressources téléchargeables suivent la même politique RLS.

app/[locale]/courses/[id]/page.tsx Lignes 145-168

```
146: let isPaid = enrollment?.payment_status === "paid";
148: if (
149:   !isPaid &&
150:   enrollment?.chargily_checkout_id &&
151:   (payment === "success" || enrollment.payment_status === "pending")
152: ) {
153:   const verified = await verifyChargilyPayment(
154:     enrollment.id,
155:     enrollment.chargily_checkout_id,
156:   );
157:   if (verified) isPaid = true;
158: }
160: // Fetch lessons
161: const { data: lessons, error: lessonsError } = await supabase
162:   .from("lessons")
163:   .select("id, title, description, duration, order_index, is_free")
164:   .eq("course_id", id)
165:   .order("order_index", { ascending: true });
167: if (lessonsError) { throw new Error(lessonsError.message);
```

GUIDE DE TEST ET DÉPANNAGE

Section 10.9 sur 14 : protocole de validation et matrice de résolutions

1 **Tunnel local ngrok** ngrok http 3000 : expose le webhook	**2** **Cartes de test Edahabia** Fournies par la doc Chargily, OTP 4-6 car.	**3** **Diagnostic logs** Console serveur, en-têtes et statuts

1. **401 Signature** : Clé secrète erronée : CHARGILY_SECRET_KEY dans .env.local doit correspondre au dashboard.
2. **Webhook non reçu** : localhost inaccessible : APP_URL avec l'URL publique ngrok (https://xxxx.ngrok-free.app).
3. **503 Non configuré** : Variables absentes : CHARGILY_SECRET_KEY dans le fichier d'environnement.
4. **Accès non débloqué** : Échec BDD : vérifier SUPABASE_SERVICE_ROLE_KEY de createAdminClient().

RÉSOLUTION DES ANOMALIES

En local, déclarer APP_URL=https://... dans .env.local.

Consulter l'historique des webhooks sur le dashboard Chargily (réponses HTTP).

Ne jamais tester avec de vraies cartes quand CHARGILY_MODE = 'test'.

docs/handouts/intern-handbook.html Diagnostic

```
Symptome constate       | Cause technique            | Solution immediate
------------------------+----------------------------+---------------------------
Webhook retourne 401    | Signature HMAC inconnue    | Synchroniser CHARGILY_SECRET_KEY
Webhook timeout (504)   | Requete base trop lente    | Verifier connexion Supabase
Inscription introuvable | Metadata checkout_id absent| Valider l'upsert prealable
Montant incoherent      | Lecture client interdite   | Le prix vient de courses.price
```

SÉCURITÉ DU SYSTÈME ET DU PAIEMENT

Section 11 sur 14 : checklist de sécurité et modèle de menace

1 Zéro secret client Clé Chargily serveur strict	**2 Signature HMAC** timingSafeEqual	**3 Prix côté BDD** courses.price infalsifiable	**4 Politiques RLS** Étudiant ne s'écrit pas 'paid'

1. **Zéro clé secrète côté client** : CHARGILY_SECRET_KEY et SUPABASE_SERVICE_ROLE_KEY absentes du bundle JS.
2. **Vérification HMAC systématique** : 401 immédiat si la signature ne coïncide pas.
3. **Tarification 100 % serveur** : Montant lu dans courses ou workshops juste avant l'appel passerelle.
4. **Zéro stockage bancaire** : Conformité PCI-DSS : aucun numéro de carte, expiration ni CVV.

RÈGLE ABSOLUE DE SÉCURITÉ

Un pirate modifiant le JSON dans son navigateur paiera le montant réel.

secret_guard.sh fait échouer le build si une clé privée est commitée.

Sessions : cookies HttpOnly chiffrés, inaccessibles au JavaScript client.

scripts/secret_guard.sh Lignes 15-45

```
15: # The Supabase anon key is deliberately NOT a finding. It is compiled
17: # into the JavaScript bundle by design and is protected by row-level
18: # security rather than by secrecy. What it looks for is the
19: # service-role key, which bypasses RLS entirely and must never be in
20: # this app at all.
26: # Files that legitimately describe secrets without containing any.
27: EXCLUDE_RE='^(\.env\.example|scripts/secret_guard\.sh|CONTRIBUTING\.md|docs/)'
29: # name=<32+ chars> where the name itself says it is a secret.
37: ASSIGNMENT_RE='(SECRET|TOKEN|PASSWORD|PRIVATE_KEY|SERVICE_ROLE|API_KEY)...'
41: VENDOR_RE='(sb_secret_...|-----BEGIN ... PRIVATE KEY-----|eyJ...)'
43: fail=0
```

DÉPLOIEMENT VERCEL ET PRODUCTION

Section 12 sur 14 : passage en production et supervision

1. Variables prod	2. Webhook config	3. Domaine et SSL	4. Test en direct
CHARGILY_MODE = live	URL HTTPS déclarée	evolve-academy.dz, TLS actif	Micro-transaction réelle

1. **Variables Vercel** : Injecter CHARGILY_SECRET_KEY, NEXT_PUBLIC_SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY.
2. **URL de webhook** : https://evolve-academy.dz/api/webhooks/chargily, déclarée dans le dashboard Chargily.
3. **Microtransaction réelle** : Paiement test (ex. 100 DZD) avec une carte Edahabia valide.
4. **Plan de secours** : Rollback en un clic vers la version Vercel précédente.

CHECKLIST DE MISE EN PRODUCTION

Vérifier que HSTS est appliqué pour forcer le HTTPS.

Consulter les logs Vercel Functions pour le temps de réponse du webhook.

En cas d'anomalie réseau, le fallback synchrone prend le relais.

.env.example Lignes 1-35

```
10: NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
13: NEXT_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key
15: # Server-only service role key (NEVER expose to browser)
16: SUPABASE_SERVICE_ROLE_KEY=your-supabase-service-role-key
21: # Operating mode: 'test' for Sandbox, 'live' for Real Production
22: CHARGILY_MODE=test
24: # Chargily Secret Key (server-side only; test_sk_ or live_sk_)
25: CHARGILY_SECRET_KEY=test_sk_your_chargily_secret_key
27: # Optional Chargily Public / API Key
28: CHARGILY_API_KEY=test_pk_your_chargily_public_key
30: # Optional API URL override
31: # CHARGILY_API_URL=https://pay.chargily.net/test/api/v2
```

DÉBOGAGE ET COMMANDES UTILES

Section 13 sur 14 : exploitation locale et FAQ des erreurs

1 **pnpm run dev** localhost:3000	2 **bash scripts/local_ci.sh** Tous les guards	3 **pnpm run lint:fix** Correction Biome	4 **pnpm run typecheck** TypeScript strict

1. **Erreur RLS : lignes vides** : L'utilisateur n'est pas connecté ou la politique SELECT exige un statut publié.
2. **Erreur 401 webhook** : Vérifier que CHARGILY_SECRET_KEY correspond à l'environnement actif.
3. **Erreur 500 route API** : Variables manquantes : NEXT_PUBLIC_SUPABASE_URL et clés de service.
4. **Erreur d'hydratation SSR** : Fréquent sur les dates : formateurs stables ou useEffect.

MAINTENANCE QUOTIDIENNE

Lancer typecheck et guards avant d'ouvrir une pull request.

Le portail tourne sur Node.js 20+ avec pnpm v10.

Voir scripts/local_ci.sh pour l'enchaînement exact des vérifications CI.

scripts/local_ci.sh Lignes 1-35

```
3: # Everything that must pass before a push.
5: #   make ci-local (or: bash scripts/local_ci.sh)
7: # Ordered so the cheapest check fails first. The four greps run in
9: # under a second, so a boundary violation or a hex colour is reported
10: # long before the slow typecheck and build.
13: # It does not stop at the first failure. Being told about three
14: # problems in one run beats three round trips.
16: set -uo pipefail
21: failed=()
23: step() { # label command...
24:   local label="$1"; shift
26:   if "$@" > /tmp/evolve-web-ci.log 2>&1; then
27:     echo "  OK $label"
28:   else
29:     echo "  KO $label"
31:     failed+=("$label")
```

AUDIT DU CODE ET ÉCARTS CONSTATÉS

Section 14 sur 14 : dette technique et plan de découpage

1 **Fichiers > 300 lignes** dashboard.ts, checkout/route.ts : 5 hors limite	2 **Zone d'alerte** HorizontalCourseSection (269 l), WorkshopVideoPlayer (269 l)	3 **Recommandations** Découper les routes hybrides, isoler les stats

1. **lib/data/dashboard.ts (694 lignes)** : Fichier le plus volumineux : requêtes d'agrégation d'accueil. À scinder par domaine.
2. **app/api/checkout/route.ts (382 lignes)** : Cours, ateliers, plafonds et rate limiter. À modulariser.
3. **messages/useMessagingClient.ts (570 lignes)** : Polling et gestion des salons. À décomposer en sous-hooks.
4. **Conformité globale** : Zéro secret commité, typage strict respecté, aucune fuite de clés.

BILAN DE L'AUDIT TECHNIQUE

component_size_guard.sh vise les fichiers .tsx de components/ et app/.

Les fichiers listés sont à traiter en priorité au prochain sprint de refactorisation.

Toutes les recommandations respectent CONTRIBUTING.md.

scripts/component_size_guard.sh Lignes 1-30

```
3: # No component or page file over 300 lines.
5: # The number is arbitrary; having a number is not. A 700-line page
6: # component is not reviewable - a reviewer skims it, approves it, and
7: # the defect ships. It is also unmergeable when two people touch it in
8: # the same sprint.
10: # The fix is always the same and always available: pull a section out
11: # into components/. If a file is 310 lines of genuinely irreducible
12: # markup, say so in the pull request and raise the limit in the same
13: # diff. What is not allowed is an exemption list that grows quietly.
19: LIMIT=300
20: WARN=250
23: while IFS= read -r file; do
25:   if [ "$lines" -gt "$LIMIT" ]; then
```

INDEX EXHAUSTIF DES FICHIERS (PARTIE 1)

Annexe A · Section 14 sur 14

Chemin du fichier	Rôle
.env.example	Configuration
.env.local	Configuration
.gitignore	Configuration
.markdownlint-cli2.jsonc	Configuration
.markdownlint.json	Configuration
CONTRIBUTING.md	Configuration
Makefile	Configuration
app/[locale]/ateliers/AteliersBrowser.tsx	Composant / route
app/[locale]/ateliers/CategoryFilter.tsx	Composant / route
app/[locale]/ateliers/SearchBar.tsx	Composant / route
app/[locale]/ateliers/WorkshopCard.tsx	Composant / route
app/[locale]/ateliers/[id]/WorkshopJoinButton.tsx	Composant / route
app/[locale]/ateliers/[id]/page.tsx	Page
app/[locale]/ateliers/loading.tsx	Chargement
app/[locale]/ateliers/page.tsx	Page
app/[locale]/community/CommunityFilters.tsx	Composant / route
app/[locale]/community/SubmitProjectForm.tsx	Composant / route
app/[locale]/community/[id]/Comments.tsx	Composant / route
app/[locale]/community/[id]/EditProjectDetailsFields.tsx	Composant / route
app/[locale]/community/[id]/EditProjectForm.tsx	Composant / route
app/[locale]/community/[id]/EditProjectImageField.tsx	Composant / route
app/[locale]/community/[id]/EditProjectLinksFields.tsx	Composant / route
app/[locale]/community/[id]/LikeButton.tsx	Composant / route
app/[locale]/community/[id]/ProjectActions.tsx	Composant / route
app/[locale]/community/[id]/ProjectActionsBar.tsx	Composant / route
app/[locale]/community/[id]/ProjectContent.tsx	Composant / route
app/[locale]/community/[id]/ProjectHeader.tsx	Composant / route
app/[locale]/community/[id]/edit/page.tsx	Page
app/[locale]/community/[id]/page.tsx	Page
app/[locale]/community/[id]/useEditProjectForm.ts	Hook
app/[locale]/community/loading.tsx	Chargement
app/[locale]/community/page.tsx	Page
app/[locale]/community/useSubmitProjectForm.ts	Hook
app/[locale]/courses/[id]/checkout/page.tsx	Page
app/[locale]/courses/[id]/edit/page.tsx	Page
app/[locale]/courses/[id]/lessons/[lessonId]/edit/page.tsx	Page
app/[locale]/courses/[id]/lessons/[lessonId]/page.tsx	Page
app/[locale]/courses/[id]/loading.tsx	Chargement
app/[locale]/courses/[id]/page.tsx	Page
app/[locale]/dashboard/DashboardClient.tsx	Composant / route
app/[locale]/dashboard/loading.tsx	Chargement
app/[locale]/dashboard/page.tsx	Page
app/[locale]/disciplines/page.tsx	Page
app/[locale]/error.tsx	Composant / route
app/[locale]/formations/FormationsBrowser.tsx	Composant / route
app/[locale]/formations/loading.tsx	Chargement
app/[locale]/formations/page.tsx	Page
app/[locale]/layout.tsx	Composant / route
app/[locale]/messages/AvailableContactsPanel.tsx	Composant / route
app/[locale]/messages/ChatHeader.tsx	Composant / route
app/[locale]/messages/ConversationSidebar.tsx	Composant / route
app/[locale]/messages/MessageComposer.tsx	Composant / route
app/[locale]/messages/MessagesThread.tsx	Composant / route
app/[locale]/messages/MessagingClient.tsx	Composant / route
app/[locale]/messages/NewChatModal.tsx	Composant / route
app/[locale]/messages/loading.tsx	Chargement
app/[locale]/messages/page.tsx	Page
app/[locale]/messages/useAvailableContacts.ts	Hook
app/[locale]/messages/useMessagingClient.ts	Hook
app/[locale]/not-found.tsx	Composant / route
app/[locale]/page.tsx	Page
app/[locale]/profile/page.tsx	Page

INDEX EXHAUSTIF DES FICHIERS (PARTIE 2)

Annexe B · Section 14 sur 14 · Fin du guide technique, réf. EV-2026-002

Chemin du fichier	Rôle
components/dashboard/DashboardCommunitySection.tsx	Composant
components/dashboard/DashboardCoursesGrid.tsx	Composant
components/dashboard/DashboardFilterTabs.tsx	Composant
components/dashboard/DashboardGuestView.tsx	Composant
components/dashboard/DashboardHeroBanner.tsx	Composant
components/dashboard/DashboardRecommendationsSection.tsx	Composant
components/dashboard/DashboardResumeBanner.tsx	Composant
components/dashboard/DashboardStatsGrid.tsx	Composant
components/dashboard/DashboardWorkshopsSection.tsx	Composant
components/dashboard/types.ts	Types
components/formations/FormationCard.tsx	Composant
components/formations/Formationcurriculum.tsx	Composant
components/formations/Formationheader.tsx	Composant
components/formations/FormationsCommunityCta.tsx	Composant
components/formations/FormationsCompletionAlert.tsx	Composant
components/formations/FormationsHero.tsx	Composant
components/formations/FormationsMobileCategories.tsx	Composant
components/formations/FormationsSearchBar.tsx	Composant
components/formations/FormationsSidebar.tsx	Composant
components/formations/i18n.ts	Composant
components/formations/types.ts	Types
components/hero/HeroCarouselControls.tsx	Composant
components/hero/HeroContent.tsx	Composant
components/hero/HeroSocialProof.tsx	Composant
components/hero/SaveCourseButton.tsx	Composant
components/hero/types.ts	Types
components/home/ContinueLearningSection.tsx	Composant
components/home/EnrollmentPathSection.tsx	Composant
components/lesson/CourseLessonsList.tsx	Composant
components/lesson/useLessonProgress.ts	Hook
components/lesson/youtube.ts	Composant
components/navbar/DesktopNav.tsx	Composant
components/navbar/MobileNav.tsx	Composant
components/navbar/types.ts	Types
components/profile/PreferencesFormFields.tsx	Composant
components/profile/PreferencesInterestsPicker.tsx	Composant
components/profile/preferencesConstants.ts	Composant
components/workshop/WorkshopCertificateSection.tsx	Composant
components/workshop/WorkshopEnrolledView.tsx	Composant
components/workshop/WorkshopIncludedFeatures.tsx	Composant
components/workshop/WorkshopProjectSubmission.tsx	Composant
components/workshop/WorkshopVideoPlayer.tsx	Composant
components/workshop/types.ts	Types
components/workshop/useWorkshopProjectSubmission.ts	Hook
docs/backup-restore-guide.md	Documentation
docs/handouts/intern-handbook.html	Documentation
docs/handouts/manager-brief.html	Documentation
docs/handouts/print.css	Documentation
docs/handouts/render.sh	Documentation
docs/week-4-foundations.md	Documentation
i18n/navigation.ts	i18n
i18n/request.ts	i18n
i18n/routing.ts	i18n
lib/analytics/log-search.ts	Utilitaire
lib/apiClient.ts	Utilitaire
lib/community-directory.ts	Utilitaire
lib/course-completion.ts	Utilitaire
lib/data/assignments.ts	Accès données
lib/data/categories.ts	Accès données
lib/data/course-payment.ts	Accès données